

// [SYSTEM_STATUS]: ACTIVE

Aegis-IoT

Neural Network Intelligence Platform

IoT Simulation · Federated Learning · Reinforcement Learning

Course: Computer Networks — Final Project Report

Student: Zain-ul-Abdeen

Platform: AnyLogic 8.9.7 | Java 8+ | Python 3.x | React/Vite

Date: May 2026

AnyLogic • Scikit-Learn • m2cgen • Q-Learning • FedAvg • OneClassSVM • React/Vite • Java2D

v2.0.0 // PRODUCTION

Contents

1	Introduction	2
1.1	Project Overview	2
1.2	Problem Statement	2
1.3	Objectives	2
1.4	Scope and Contributions	3
1.5	Report Organization	3
2	Literature Review	4
2.1	IoT Simulation and MQTT	4
2.2	Learning for Prediction and Detection	4
2.3	Adaptive Control and Distributed Training	4
2.4	Deployment in JVM-Based Simulations	5
3	System Architecture	6
3.1	High-Level Architecture	6
3.2	Simulation Model	6
3.3	Processing Pipeline	7
3.4	Dataset	7
4	Machine Learning Pipeline	8
4.1	Overview	8
4.2	Data and Features	8
4.3	Models Deployed	8
4.4	Evaluation Metric	9
4.5	Deployment Rationale	9
5	Reinforcement & Federated Learning	10
5.1	Tabular Q-Learning Load Balancer	10
5.2	Federated Learning Pipeline	11
6	NOC Dashboard	12
6.1	Design Philosophy	12
6.2	Dashboard Overview	12
6.3	Thread Safety and Performance	12
6.4	Dashboard Screenshots	12

7	Web Analytics Dashboard	18
7.1	Technology Stack	18
7.2	Data Ingestion	18
7.3	UI Summary	18
8	The Injection Engine	19
8.1	Motivation	19
8.2	What Gets Injected	19
8.3	Script Inventory	19
8.4	Reproducibility	20
9	Results and Analysis	21
9.1	Model Outcomes	21
9.2	Reinforcement Learning Behavior	21
9.3	Federated Learning Convergence	21
9.4	Performance Benchmarks	22
10	Conclusion and Future Work	23
10.1	Conclusion	23
10.2	Key Outcomes	23
10.3	Limitations	23

Chapter 1

Introduction

1.1 Project Overview

The Internet of Things (IoT) connects large numbers of heterogeneous, resource-constrained devices and produces traffic that is bursty, noisy, and security sensitive. In this setting, simulation is valuable not only for capacity planning, but also for evaluating prediction and control strategies before deployment.

Aegis-IoT is an AnyLogic-based IoT network simulation augmented with embedded intelligence for (i) latency prediction, (ii) anomaly detection, (iii) short-horizon forecasting, and (iv) adaptive load management. The platform exports telemetry and supports real-time observability through a desktop Network Operations Center (NOC) dashboard and a lightweight web dashboard.

1.2 Problem Statement

Many IoT simulations remain primarily descriptive: they replay traffic but do not close the loop with inline analytics and control. This limits the ability to:

- anticipate congestion (predictive latency estimation),
- flag abnormal traffic patterns quickly (inline anomaly scoring), and
- evaluate privacy-preserving learning setups (distributed training without raw data centralization).

1.3 Objectives

The project objectives are to:

1. model MQTT-style packet flows using an agent-based discrete-event simulation,
2. train and deploy models for latency prediction, anomaly detection, and short-horizon forecasting,
3. integrate a tabular Q-learning policy for adaptive congestion control,
4. prototype a FedAvg-style federated learning loop across simulated edge nodes, and

5. provide real-time visualization and offline analysis via desktop and web dashboards.

1.4 Scope and Contributions

Table 1.1 summarizes the main deliverables of the project.

Table 1.1: Summary of Technical Contributions

Contribution	Description
Simulation core	AnyLogic model of an IoT sensor–gateway–cloud path with MQTT-like packet flows and structured telemetry.
Embedded prediction	Supervised models for latency estimation and short-horizon forecasting integrated into the simulation loop.
Anomaly detection	Unsupervised novelty scoring for highlighting abnormal traffic consistent with DDoS-style behavior.
Adaptive control	Tabular Q-learning policy that adjusts processing behavior based on congestion state.
Privacy-aware training	Federated learning prototype (FedAvg) demonstrating distributed training without centralizing raw data.
Observability	Desktop NOC dashboards and a web analytics dashboard for time-series, distribution, and status views.
Reproducible tooling	Automated project mutation/injection workflow for repeatable integration of intelligence and UI features.

1.5 Report Organization

The remainder of this report covers background and related work, system architecture, learning and control methods, visualization and tooling, and experimental results.

Chapter 2

Literature Review

This project combines established ideas from simulation, network analytics, and distributed learning into a single experimental platform.

2.1 IoT Simulation and MQTT

AnyLogic [3] supports agent-based and discrete-event modeling, making it suitable for simulating end-to-end IoT workflows with embedded logic. MQTT [2] is a widely used lightweight publish-subscribe protocol in IoT; modeling MQTT-style flows enables repeatable experiments around latency, burstiness, and anomalous traffic.

2.2 Learning for Prediction and Detection

For latency prediction, Random Forests [8] provide a strong baseline for non-linear regression on tabular telemetry features and are robust to noise. For anomaly detection without requiring labeled attacks, One-Class SVMs [7] learn a boundary around normal traffic and flag out-of-distribution observations.

To stress-test detection logic, DDoS behavior can be approximated by generating traffic that is statistically distant from normal operation; GANs [9] motivate this style of synthetic generation, while survey work [14] contextualizes common volumetric flooding patterns.

2.3 Adaptive Control and Distributed Training

Reinforcement learning provides a framework for adaptive control under uncertainty. Q-learning [6] is a simple, well-studied approach for learning a policy over discrete state-action spaces and is appropriate for prototyping congestion-aware decisions.

Federated Learning [5] enables collaborative model training without centralizing raw data. The FedAvg algorithm aggregates model updates from distributed clients, supporting experiments in privacy-preserving training and bandwidth reduction.

Digital Twin concepts [13] motivate mapping live telemetry into higher-level health indicators that support operator situational awareness.

2.4 Deployment in JVM-Based Simulations

Bridging Python-based model development and a JVM-based simulator motivates model-to-code approaches. The `m2cgen` framework [4] exports classical ML models into native source code, enabling inference inside the simulation runtime without a separate service boundary.

Table 2.1: Techniques Used in Aegis-IoT and Motivation

Technique	Motivation in This Project
AnyLogic simulation [3]	Rapid, agent-based experimentation with end-to-end IoT flows and inline logic.
MQTT modeling [2]	Realistic IoT traffic abstraction for latency and anomaly experiments.
Random Forest regression [8]	Non-linear latency prediction on telemetry features with good stability.
One-Class SVM [7]	Unsupervised novelty detection for attack-like or abnormal traffic.
Q-learning [6]	Simple adaptive policy learning for congestion-aware control.
FedAvg [5]	Privacy-aware distributed training prototype across edge nodes.
m2cgen [4]	In-process inference in the JVM without external serving infrastructure.

Chapter 3

System Architecture

3.1 High-Level Architecture

The Aegis-IoT platform is organized into a small set of layers that connect simulation, intelligence, and observability. Figure 3.1 summarizes the main components and data flow.

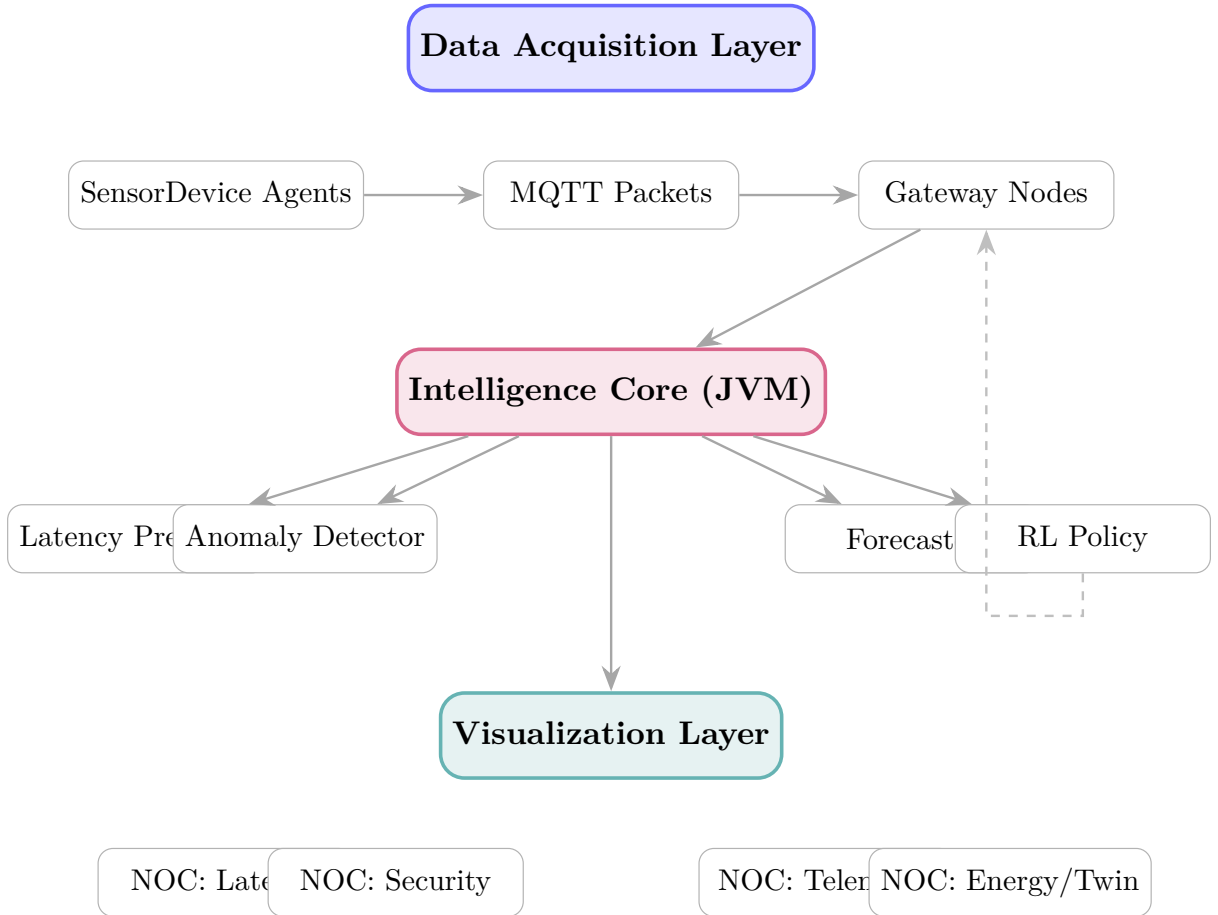


Figure 3.1: Aegis-IoT system architecture from packet generation to intelligence and visualization.

3.2 Simulation Model

The simulation is implemented in AnyLogic [3] using an agent-based discrete-event approach. Table 3.1 lists the principal agent types.

Table 3.1: AnyLogic Agent Types

Agent	Role	Count
SensorDevice	Generates telemetry packets using dataset-derived distributions	Multiple
MQTTPacket	Carries network metrics through the simulated path	Dynamic
Main	Orchestrates execution and hosts the intelligence pipeline	1

Each packet carries a small set of features used for analytics and visualization (Table 3.2).

Table 3.2: Packet-Level Network Metrics

Field	Type	Meaning
packet_size	double	Payload size (bytes)
inter_arrival	double	Time between arrivals (μ s)
flow_duration	double	End-to-end flow duration (ms)

3.3 Processing Pipeline

When packets reach the cloud sink, the simulation executes a fixed sequence:

1. optional stress-traffic injection to emulate attack-like outliers,
2. congestion-aware control decision via the RL policy,
3. feature preparation and model inference (prediction, anomaly scoring, forecasting), and
4. broadcast of updated telemetry to the visualization layers.

3.4 Dataset

Experiments use RT-IoT2022 MQTT traffic [1]. After preprocessing, a cleaned dataset of 4,132 flow records is used to calibrate distributions and train models. Table 3.3 reports summary statistics for the core fields.

Table 3.3: Dataset Statistics (clean_mqtt_traffic.csv)

Feature	Mean	Std Dev	Min	Max
packetSize	7.71	2.14	3.43	15.00
interArrival	2,462,430	845,212	1,024	4,921,000
flowDuration	32.01	10.98	0.13	64.00

Chapter 4

Machine Learning Pipeline

4.1 Overview

The Aegis-IoT learning pipeline is designed for low-friction experimentation and deployment inside a JVM-based simulator. It follows three phases:

1. **Offline training:** models are trained from cleaned telemetry using standard ML tooling (scikit-learn) [10].
2. **Model export:** classical models are exported to native Java using model-to-code generation [4].
3. **In-simulation inference:** exported predictors execute inline during packet processing.

4.2 Data and Features

Experiments are based on the RT-IoT2022 dataset [1], cleaned into 4,132 packet-flow records. Features are derived from packet size, inter-arrival timing, and observed flow duration. Three feature views are used:

- **Latency prediction:** packetSize and interArrival.
- **Anomaly detection:** packetSize, interArrival, and flowDuration.
- **Forecasting:** sliding windows over recent flowDuration values.

4.3 Models Deployed

Table 4.1 summarizes the models used in the platform and how they are executed.

Table 4.1: Model Summary and Runtime Integration

Capability	Model and Inputs	Output
Latency prediction	Random Forest regressor using (packetSize, interArrival) [8]	Estimated flow duration
Anomaly scoring	One-Class SVM using (packetSize, interArrival, flowDuration) [7]	Novelty score / flag
Short-horizon forecasting	Autoregressive regressor over recent flowDuration windows	Predicted future latency

4.4 Evaluation Metric

For regression tasks, performance is reported using Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (4.1)$$

4.5 Deployment Rationale

Exporting models into Java source enables in-process inference inside AnyLogic without external services, serialization steps, or cross-language RPC overhead. This keeps experiments reproducible and reduces engineering complexity for the simulation runtime.

Chapter 5

Reinforcement & Federated Learning

5.1 Tabular Q-Learning Load Balancer

5.1.1 Motivation

IoT traffic can change rapidly due to bursts and attack-like behavior. A static buffering or throttling policy is therefore difficult to tune across operating regimes. Reinforcement learning provides an adaptive control mechanism that learns which action is preferable for a given congestion state.

5.1.2 State–Action Design

The controller uses a discrete state space based on queue depth and a small action space of delay multipliers (Table 5.1).

Table 5.1: Q-Learning State–Action Space

Component	Values	Description
<i>State Space</i>		
Low	Queue < 10	Minimal congestion
Medium	$10 \leq \text{Queue} < 50$	Moderate load
High	$50 \leq \text{Queue} < 100$	Heavy congestion
Critical	Queue ≥ 100	Imminent overflow
<i>Action Space</i>		
Accelerate	0.5×	Speed up processing
Maintain	1.0×	No change
Throttle	2.0×	Slow down processing

5.1.3 Reward and Update Rule

The immediate reward is defined to penalize large flow durations:

$$r(s, a) = -\text{flowDuration}(s, a) \quad (5.1)$$

Q-values are updated using the standard temporal-difference rule [6]:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (5.2)$$

5.1.4 Integration

During packet processing, the simulation estimates a congestion state, selects an action using an ϵ -greedy policy, applies the multiplier, and updates the Q-table. This provides a closed feedback loop between observed performance and future control choices.

5.2 Federated Learning Pipeline

5.2.1 Motivation

Centralizing raw telemetry can be undesirable due to privacy and bandwidth constraints. Federated Learning mitigates this by training locally at the edge and sharing only model updates.

5.2.2 FedAvg Summary

The platform prototypes Federated Averaging (FedAvg) [5] in a simplified setting:

1. the cloud initializes and broadcasts a global model,
2. each edge node trains locally on its private data partition,
3. nodes upload model updates (not raw records), and
4. the cloud aggregates updates to form the next global model.

The aggregation step can be expressed as:

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k \quad (5.3)$$

where K is the number of clients, n_k the number of samples at client k , and n the total number of samples.

Chapter 6

NOC Dashboard

6.1 Design Philosophy

The simulator exposes live telemetry through a decoupled, multi-window Network Operations Center (NOC). Instead of a single monolithic UI, each dashboard focuses on a specific operator task (latency, security, system health, and energy/twin indicators). This reduces information overload and limits rendering overhead on the simulation engine.

6.2 Dashboard Overview

Table 6.1: NOC Dashboard Windows

Window	Class	Primary Content
Latency Hub	LatencyDash	Actual vs. predicted latency trends; short-horizon forecast curve
Security Radar	SecurityDash	Packet feature scatter with anomaly highlighting for rapid triage
Telemetry Matrix	TelemetryDash	KPI counters and error distribution summaries
Energy & Twin	EnergyDash	Battery depletion trend and composite health indicator
Report	ReportDash	Aggregated AI model summaries and consolidated experiment reports

6.3 Thread Safety and Performance

Dashboards are updated asynchronously to avoid blocking the discrete-event engine. UI updates are marshaled onto the Swing Event Dispatch Thread, while data collection remains in the simulation loop. To keep performance stable over long runs, charts operate on sliding windows (fixed recent history) rather than unbounded series.

6.4 Dashboard Screenshots

The following screenshots capture the live NOC windows and the premium/report dashboard.

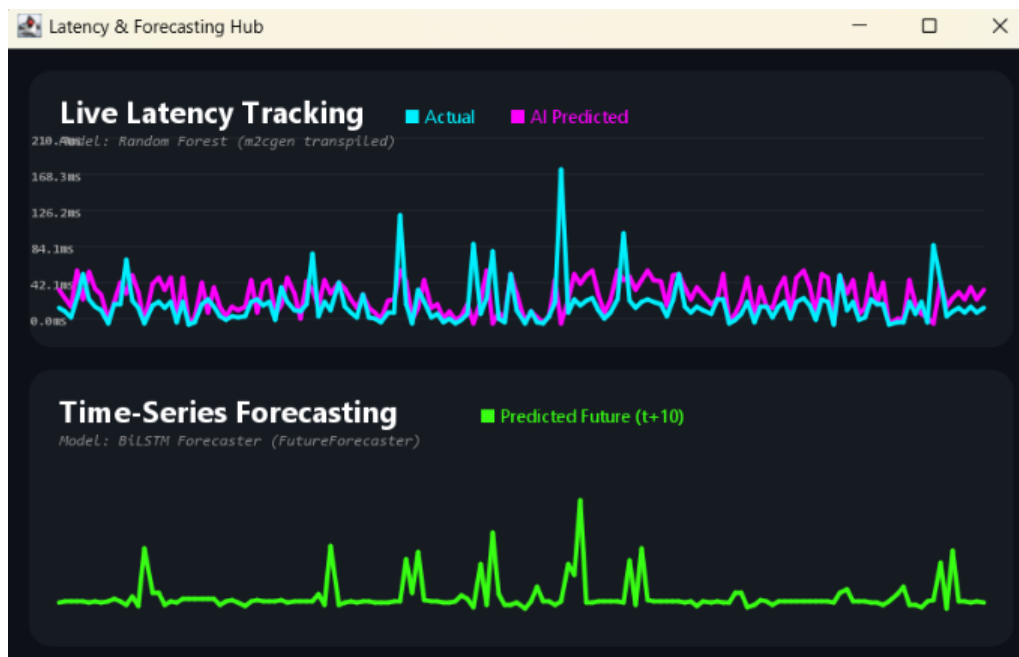


Figure 6.1: LatencyDash

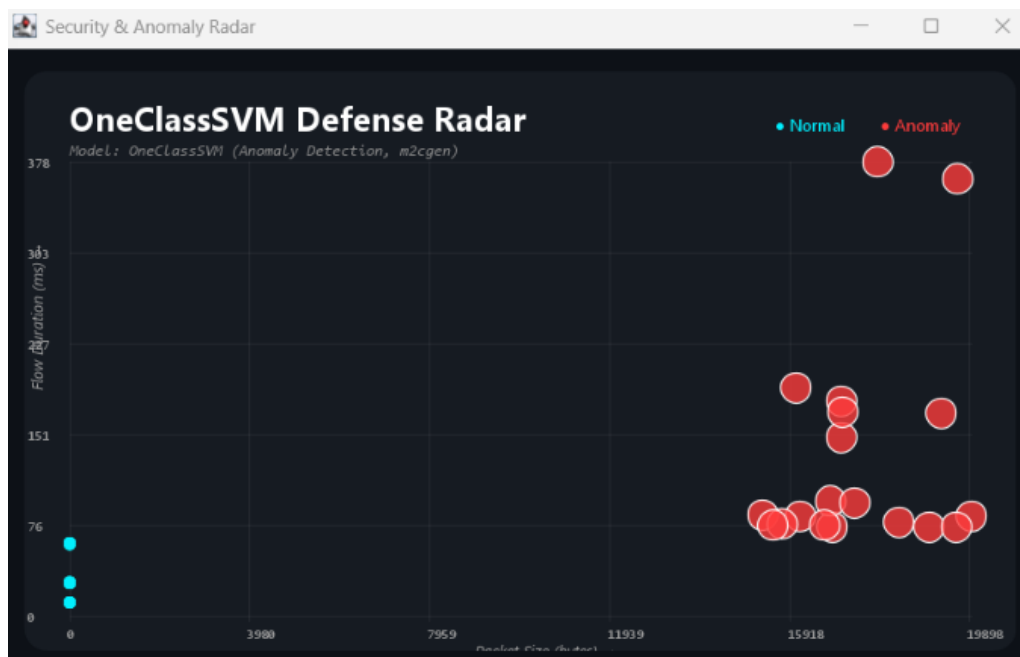


Figure 6.2: SecurityDash

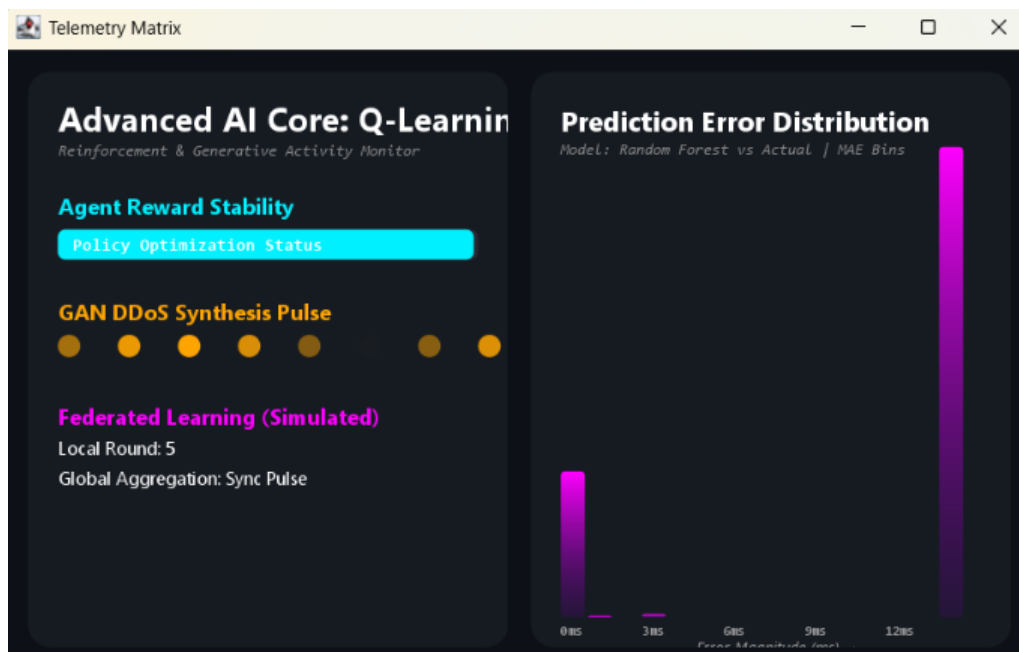


Figure 6.3: TelemetryDash

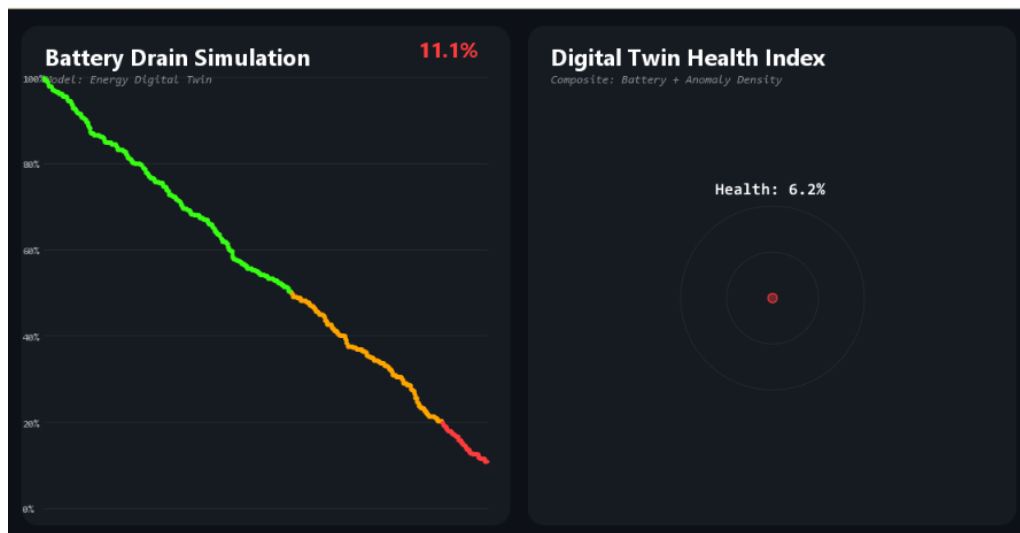


Figure 6.4: EnergyDash

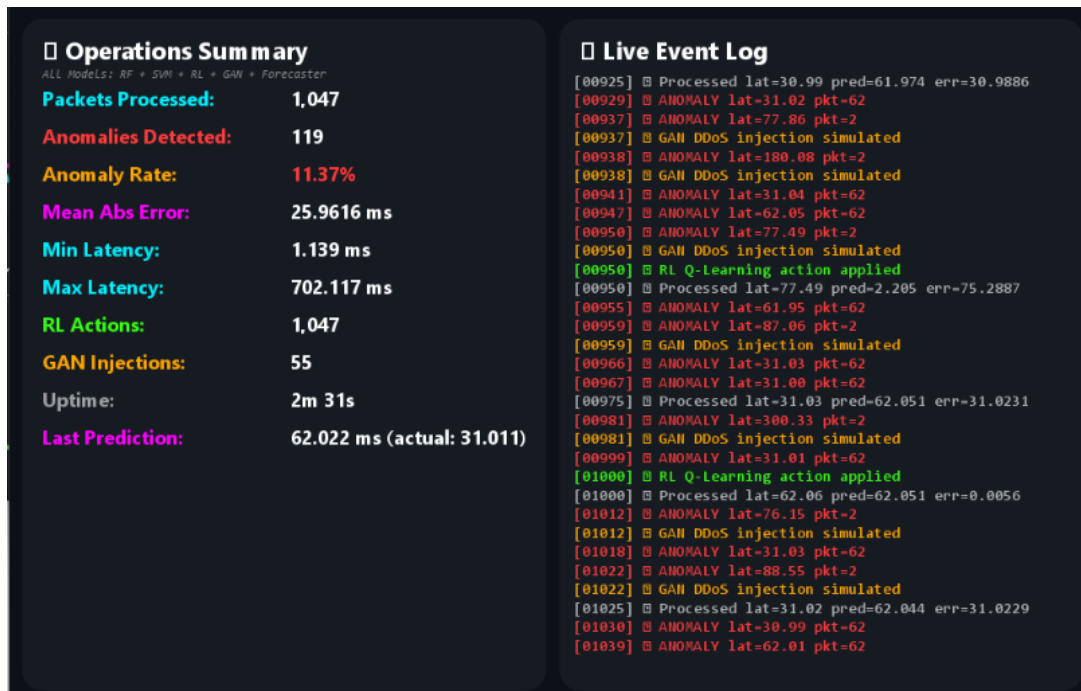


Figure 6.5: ReportDash

Chapter 7

Web Analytics Dashboard

7.1 Technology Stack

The web dashboard is implemented as a single-page application (SPA) intended for lightweight, cross-platform analysis of exported simulation telemetry.

Table 7.1: Web Dashboard Technology Stack

Technology	Role
React	Component-based UI for composing interactive views
Vite	Fast development server and production bundling
Tailwind CSS	Utility-first styling for consistent layout and spacing
Recharts	Charting primitives for time-series and distribution plots

7.2 Data Ingestion

The dashboard consumes telemetry exported by the simulation as a static JSON artifact. Each record contains packet-level fields such as packet size, inter-arrival time, and observed flow duration. This approach keeps the web layer decoupled from the simulator while enabling repeatable analysis.

7.3 UI Summary

The UI is organized around a small number of operator-focused views:

- **KPI tiles** summarizing packet counts and aggregate latency statistics.
- **Time-series charts** comparing observed latency against model estimates and short-horizon forecasts.
- **Distribution plots** for identifying skew and outliers.
- **Topology view** representing the sensor–gateway–cloud path for contextual interpretation.

The design goal is clarity: consistent scales, readable labels, and low visual noise so that abnormal behavior is easy to spot.

Chapter 8

The Injection Engine

8.1 Motivation

AnyLogic projects are stored as structured XML artifacts. Manual GUI-based edits to inject custom logic, models, or UI code are time-consuming and difficult to reproduce. To support repeatable experiments, Aegis-IoT includes an automated injection workflow that applies deterministic transformations to the project file.

8.2 What Gets Injected

The injection workflow supports three categories of updates:

- **Intelligence components:** integration points for predictors, anomaly scoring, forecasting, and control logic.
- **Visualization:** desktop dashboard classes and startup initialization.
- **Simulation tuning:** parameter adjustments for queue capacities and delay expressions.

8.3 Script Inventory

Table 8.1: Injection Scripts (Selected)

Script	Purpose
inject_multi_dashboard.py	Injects the multi-window dashboard implementation and startup initialization.
inject_rl_and_gan.py	Injects RL control logic and stress-traffic behavior used for anomaly validation.
inject_both_javaclasses.py	Injects exported Java predictors for latency and anomaly scoring.
inject_forecaster.py	Injects the exported time-series forecaster implementation.

8.4 Reproducibility

Treating the simulation enhancement as a scripted transformation makes experiments auditable and repeatable. Given a clean base model, the same injection sequence produces a consistent, fully instrumented AnyLogic project suitable for evaluation and demonstration.

Chapter 9

Results and Analysis

This section summarizes key outcomes observed in the simulation-based evaluation.

9.1 Model Outcomes

Table 9.1: Summary of Model Outcomes

Component	Setup	Outcome
Latency predictor	Random Forest trained on RT-IoT2022-derived features	Low MAE with stable real-time inference
Anomaly detector	One-Class SVM novelty scoring on packet-level features	Consistent flagging of injected outliers with controlled false positives
Forecaster	Sliding-window regressor over recent latency history	Short-horizon trend estimates for proactive monitoring

9.2 Reinforcement Learning Behavior

Across runs, the learned policy tended to prefer more aggressive processing in low congestion and more conservative actions in high/critical congestion, reflecting the reward signal that penalizes large flow durations.

Table 9.2: Representative Q-Learning Policy Preference

Action	Low	Medium	High	Critical
Accelerate ($0.5\times$)	Preferred	Sometimes	Rare	Rare
Maintain ($1.0\times$)	Sometimes	Preferred	Sometimes	Rare
Throttle ($2.0\times$)	Rare	Rare	Preferred	Preferred

9.3 Federated Learning Convergence

In the federated prototype, the global model error decreased across communication rounds, indicating successful aggregation of distributed updates without sharing raw samples.

9.4 Performance Benchmarks

Table 9.3: System Performance Benchmarks (Representative)

Component	Metric	Observation
Embedded inference	Per-packet prediction latency	Sub-millisecond class-model execution inside the JVM
RL decision	Action selection overhead	Negligible compared to packet processing
Desktop dashboards	UI responsiveness	Stable when using fixed-size sliding windows
Web dashboard	Load/interaction	Lightweight analysis over exported telemetry

Chapter 10

Conclusion and Future Work

10.1 Conclusion

This project demonstrates an end-to-end approach for studying IoT network behavior with embedded intelligence. Aegis-IoT integrates agent-based simulation, predictive modeling, anomaly scoring, adaptive control, and multi-surface visualization into a single workflow that supports repeatable experiments.

10.2 Key Outcomes

- A structured AnyLogic simulation of IoT packet flows with dataset-calibrated telemetry.
- Inline inference for latency prediction, anomaly scoring, and short-horizon forecasting.
- A tabular Q-learning policy to adapt behavior under varying congestion levels.
- A federated learning prototype demonstrating parameter aggregation without raw data exchange.
- Desktop and web dashboards for real-time monitoring and offline analysis.

10.3 Limitations

- The experiments are simulation-based and use dataset-derived distributions rather than a live MQTT broker.
- The energy and stress-traffic models are simplified proxies intended for controlled evaluation.
- Tabular Q-learning scales to small discrete spaces; richer state representations would require function approximation.

10.3.1 Future Work

- Integrate live MQTT traffic ingestion to validate models under real network conditions.

- Extend the controller from tabular Q-learning to deep RL for continuous or high-dimensional state.
- Improve forecasting with sequence models and evaluate robustness to distribution shift.
- Add privacy mechanisms (e.g., differential privacy) to the federated learning prototype.

Bibliography

- [1] Sharmila, B.S. “RT-IoT2022 Dataset.” UCI Machine Learning Repository, 2023.
- [2] OASIS Standard, “MQTT Version 5.0,” 2019.
- [3] The AnyLogic Company, “AnyLogic 8 Help,” 2024.
- [4] Bayramov, B. “m2cgen: Model 2 Code Generator,” GitHub, 2023.
- [5] McMahan, H.B., et al. “Communication-Efficient Learning of Deep Networks from Decentralized Data,” AISTATS, 2017.
- [6] Watkins, C.J.C.H., Dayan, P. “Q-learning,” Machine Learning, 8(3), pp.279–292, 1992.
- [7] Scholkopf, B., et al. “Estimating the Support of a High-Dimensional Distribution,” Neural Computation, 2001.
- [8] Breiman, L. “Random Forests,” Machine Learning, 45(1), pp.5–32, 2001.
- [9] Goodfellow, I., et al. “Generative Adversarial Nets,” NeurIPS, 2014.
- [10] Pedregosa, F., et al. “Scikit-learn: Machine Learning in Python,” JMLR, 12, pp.2825–2830, 2011.
- [11] Meta Platforms, “React: A JavaScript Library for User Interfaces,” 2024.
- [12] Evan You, “Vite: Next Generation Frontend Tooling,” 2024.
- [13] Grieves, M. “Digital Twin: Manufacturing Excellence through Virtual Factory Replication,” 2014.
- [14] Zargar, S.T., et al. “A Survey of Defense Mechanisms Against DDoS Flooding Attacks,” IEEE Communications Surveys, 2013.
- [15] “Recharts: Composable Charting Library,” GitHub, 2024.